Exact cover search

Generated on Mon Feb 23 2026 21:11:29 for Exact cover search by Doxygen 1.9.8

Mon Feb 23 2026 21:11:29

1 Exact cover search using dancing links Knuth's algorithm 1

2 Data Structure Index 2
2.1 Data Structures . . . 2

3 File Index 2
3.1 File List . . . 2

4 Data Structure Documentation 2
4.1 element Struct Reference . . . 2
4.1.1 Detailed Description . . . 3
4.2 universe Struct Reference . . . 4
4.2.1 Detailed Description . . . 4

5 File Documentation 5
5.1 dancing_links.c File Reference . . . 5
5.1.1 Detailed Description . . . 6
5.1.2 Macro Definition Documentation . . . 6
5.1.3 Function Documentation . . . 6
5.1.4 Variable Documentation . . . 17
5.2 dancing_links.h File Reference . . . 18
5.2.1 Detailed Description . . . 19
5.2.2 Typedef Documentation . . . 19
5.2.3 Function Documentation . . . 19
5.2.4 Variable Documentation . . . 25
5.3 dancing_links.h . . . 25

Index 27

1 Exact cover search using dancing links Knuth's algorithm

Dancing links, Donald E. Knuth, Submitted on 15 Nov 2000 (see http://en.wikipedia.org/wiki/Dancing_Links, http://arxiv.org/abs/cs/0011047 and https://arxiv.org/pdf/cs/0011047v1.pdf).

In computer science, Dancing Links, also known as DLX, is the technique suggested by Donald Knuth to efficiently implement his Algorithm X. Algorithm X is a recursive, nondeterministic, depth-first, backtracking algorithm that finds all solutions to the exact cover problem.

Some of the better-known exact cover problems include tiling, the n queens problem, and Sudoku. The name Dancing Links comes from the way the algorithm works, as iterations of the algorithm cause the links to "dance" with partner links so as to resemble an "exquisitely choreographed dance."

Knuth credits Hiroshi Hitotsumatsu and Kohei Noshita with having invented the idea in 1979, but it is his paper which has popularized it.

See also http://en.wikipedia.org/wiki/Exact_cover and http://en.wikipedia.org/wiki/Exact_cover#Sudoku for more.

2 Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

element 2

universe 4

3 File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

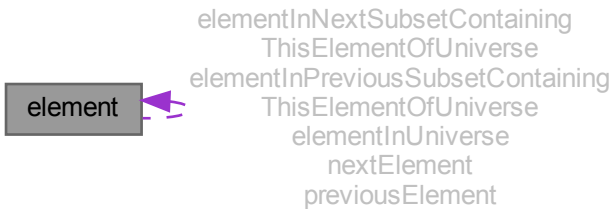
dancing_links.c 5

dancing_links.h 18

4 Data Structure Documentation

4.1 element Struct Reference

Collaboration diagram for element:



Data Fields

- char * **name**

  Either the name of the head ("|HEAD|"), or of the element of the universe, or of the subset containing the element of a subset.
- unsigned long int **size**

  Number of subsets (for head) or of subsets containing an element of the universe. Left undefined for elements of subsets.
- struct element * **previousElement**

  Link to the previous element in universe or in the subset.
- struct element * **nextElement**

  Link to the previous element in universe or in the subset.
- struct element * **elementInPreviousSubsetContainingThisElementOfUniverse**

  Link to the same element in the previous subset. Left undefined for head.
- struct element * **elementInNextSubsetContainingThisElementOfUniverse**

  Link to the same element in the next subset. Left undefined for head.
- struct element * **elementInUniverse**

  Link to the element in universe. Left undefined for head and elements in universe.

4.1.1 Detailed Description

Structure of an element (either the head or an element of the universe or an element of a subset)

There are three types of elements:

- The head is not a real element but the entry point to the elements of the universe. Its name is "|HEAD|".
- Elements of the universe. Each element of the universe is an entry point to the elements in subsets.
- Elements into subsets. An element is included in a subset when it is linked to an element of the universe.

The head and elements of universe are doubly linked as circular lists (`previousElement` and `next↩Element`). An element of the universe and subsets containing this element of the universe are doubly linked as circular lists (`elementInPreviousSubsetContainingThisElementOfUnivers` and `element↩InNextSubsetContainingThisElementOfUnivers`). The elements of a subset are doubly linked as circular lists (`previousElement` and `nextElement`).

In OOP,

- an abstract base class `A` would contain attributes `previousElement` and `nextElement` (of type `A`),
- a class `H` would describe the head of a universe and would inherit from the base class `A` extended with an attribute `size`, the total number of subsets.
- an abstract base class `B` would inherit from class `A` extended with attributes `elementInPrevious↩SubsetContainingThisElementOfUnivers` and `elementInNextSubsetContaining↩ThisElementOfUnivers` (of type B).
- a class `U` would describe elements of the universe, inherit from class `B` extended with attributes `size`, the number of subsets containing this element, and `name`, the name of the element
- a class `S` would describe elements part of subsets, inherit from class `B` extended with attributes `element↩InUnivers` (of type `U`), the element of the universe included in the subset and `name`, the name of the subset containing this element.

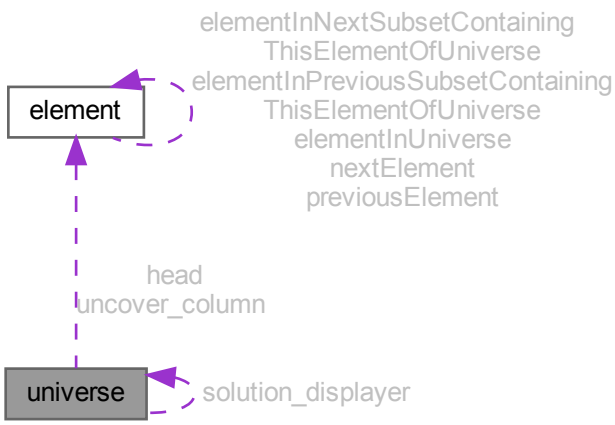
In C, all attributes are gathered into a common structure and unnecessary attributes are left undefined.

The documentation for this struct was generated from the following file:

- dancing_links.c

4.2 universe Struct Reference

Collaboration diagram for universe:



Data Fields

- struct element ∗ **head**

  The pointer to the head element that references elements and subsets.
- char ∗∗ **solution**

  Buffer to store the current solution (a list of names of subsets.)
- unsigned long **solution_length**

  Length of the buffer (number of subsets in solution.)
- struct element ∗∗ **uncover_column**

  List of subsets required in solutions (an element of this subset.)
- unsigned long **uncover_column_length**

  Number of subsets required in solutions.
- dlx_solution_displayer **solution_displayer**

  Callback function to display a solution.
- void ∗ **solution_displayer_data**

  Data usable for callback function to display a solution.
- int **unbound**

  The universe boundaries expand to the boundaries of its subsets.

4.2.1 Detailed Description

The Univers object.

Links to the first element of the universe (head), the solutions found, as well as subsets required in solutions.

The documentation for this struct was generated from the following file:

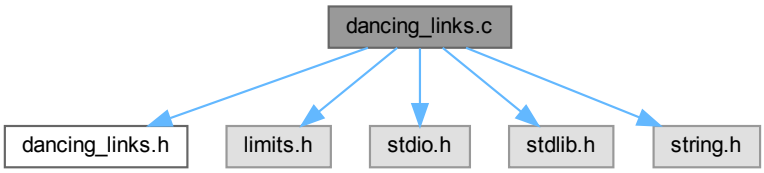
- dancing_links.c

5 File Documentation

5.1 dancing_links.c File Reference

```
#include "dancing_links.h"
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for dancing_links.c:



Data Structures

- struct element
- struct universe

Macros

- #define **OPTIMIZE_CHOICE** 1

  If set, the non-determistic choice (by `dlx_universe_choose_element`) of column is optimized heuristically.
- #define **DLX_PRINT**(...) (dlx_trace ? fprintf (stderr, __VA_ARGS__) : snprintf (0, 0, __VA_ARGS__))

  Display to terminal standard error if `dlx_trace` is set.
- #define DLX_DISPLAY_SOLUTION(universe, length, solution)

  Call the callback function `dlx_displayer` if set.

Functions

- static struct element ∗ dlx_head_get_element_by_name (struct element ∗head, const char ∗element_name)
- static int dlx_head_add_element (struct element ∗head, const char ∗name)
- static struct element ∗ dlx_head_choose_element (struct element ∗head)
- static void dlx_element_cover (struct element ∗elementInUniverse)
- static void dlx_element_uncover (struct element ∗elementInUniverse)
- static void dlx_universe_display_solutions (Universe universe, struct element ∗∗solutions)
- static unsigned long dlx_universe_search (Universe universe, struct element ∗∗solutions, unsigned long k, int one_only)
- dlx_solution_displayer dlx_displayer_set (Universe universe, dlx_solution_displayer msd, void ∗data)
- Universe dlx_universe_create (void)
- Universe dlx_universe_create (unsigned long nb_elements, const char ∗elements[ ]) __attribute__↩ ((overloadable))

- Universe dlx_universe_create (const char ∗elements, const char ∗separators) __attribute__((overloadable))
- int dlx_subset_define (Universe universe, const char ∗subset_name, unsigned long nb_elements, const char ∗elements[ ]) __attribute__((overloadable))
- int dlx_subset_define (Universe universe, const char ∗subset_name, const char ∗some_elements, const char ∗separators) __attribute__((overloadable))
- int dlx_subset_require_in_solution (Universe universe, const char ∗subset_name)
- unsigned long dlx_exact_cover_search (Universe universe, int one_only)
- void dlx_universe_destroy (Universe universe)

Variables

- int dlx_trace = 0

5.1.1 Detailed Description

Exact cover search implementation.

5.1.2 Macro Definition Documentation

DLX_DISPLAY_SOLUTION

```
#define DLX_DISPLAY_SOLUTION(
            universe,
            length,
            solution )
```

Value:

```
do {                                                                                    \
  if (universe->solution_displayer)                                                     \
    universe->solution_displayer (universe, length, solution, universe->solution_displayer_data); \
} while (0)
```

Call the callback function `dlx_displayer` if set.

5.1.3 Function Documentation

dlx_head_get_element_by_name()

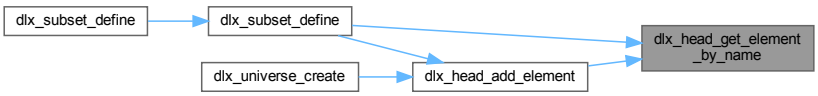
```
static struct element * dlx_head_get_element_by_name (
            struct element * head,
            const char * element_name )  [static]
```

Gets an element by its name.

Parameters

in	*head*	Univers head
in	*element_name*	Name of the element in universe to be fetched

Here is the caller graph for this function:



dlx_head_add_element()

```
static int dlx_head_add_element (
            struct element * head,
            const char * name )  [static]
```

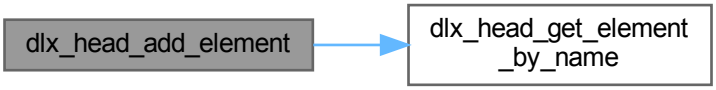
Adds an element in the universe.

Parameters

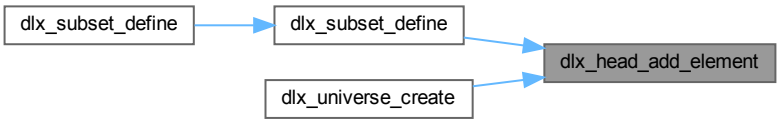
in	*head*	Univers head
in	*name*	Name of the element to be added

Initializes the element in the universe.

The head and elements of universe are doubly linked as circular lists (`previousElement` and `next↩Element`).Here is the call graph for this function:



Here is the caller graph for this function:



dlx_head_choose_element()

```
static struct element * dlx_head_choose_element (
            struct element * head )  [static]
```

Chooses an element in the universe.

Parameters

in	*head*	Univers head

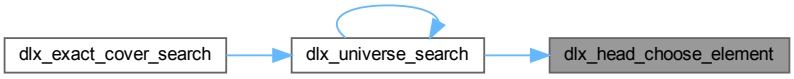
Returns

Chosen element in universe

Note

Makes use of flag OPTIMIZE_CHOICE to select the element with the minimal number of subsets that contain it (if set) or the first element, in order of the elements declared in the universe (if not).

Here is the caller graph for this function:



dlx_element_cover()

```
static void dlx_element_cover (
            struct element * elementInUniverse )  [static]
```

Removes an element and all the elements of subsets which contain this element.

Parameters

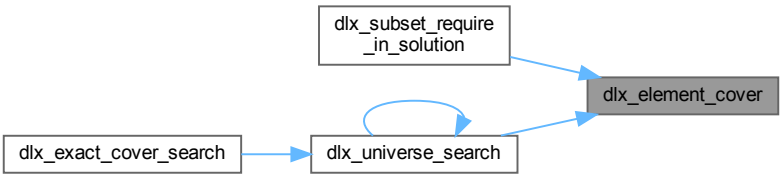
in	*elementInUniverse*	Element to be removed.

Postcondition

User must call dlx_element_uncover(struct element ∗elementInUnivers) later.

Removes the element from the universe. The elements in subsets that contains this element are also removed from

the universe. Here is the caller graph for this function:



dlx_element_uncover()

```
static void dlx_element_uncover (
            struct element * elementInUniverse )  [static]
```

Restores an element and all the elements of subsets which contain this element.

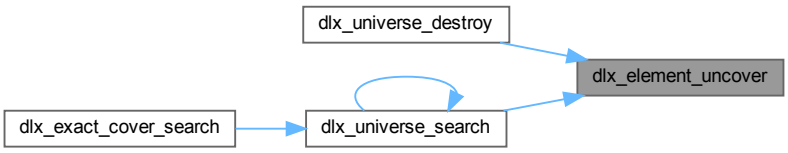
Parameters

in	*elementInUniverse*	Element to be restored.

Precondition

Use dlx_element_cover(struct element ∗elementInUnivers) first.

Here is the caller graph for this function:



dlx_universe_display_solutions()

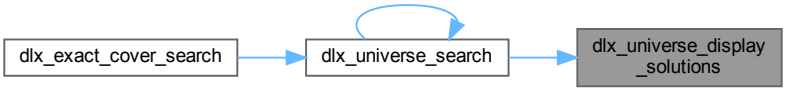
```
static void dlx_universe_display_solutions (
            Universe universe,
            struct element ** solutions )  [static]
```

Displays a solution.

Parameters

in	*universe*	Universe
in	*solutions*	Solutions to be dispplayed. Each solution is a list of elements.

Here is the caller graph for this function:



dlx_universe_search()

```
static unsigned long dlx_universe_search (
            Universe universe,
            struct element ** solutions,
            unsigned long k,
            int one_only )  [static]
```

Recursive function to search for solutions.

Parameters

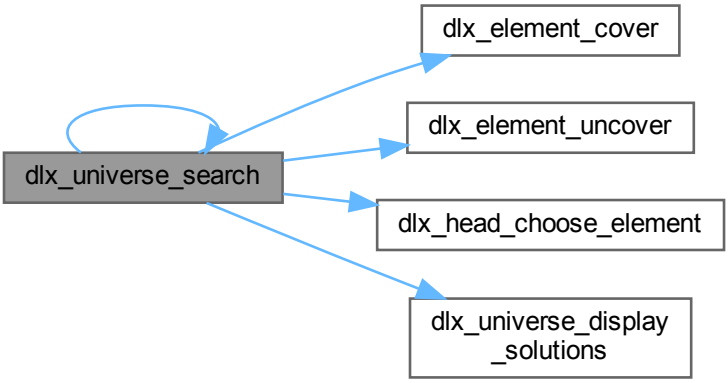
in	*universe*	Universe
in	*solutions*	Solution to be dispplayed.
in	*k*	Depth of search
in	*one_only*	If set, searches for the first solution only.

Returns

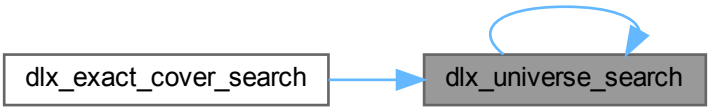
Number of solutions found.

Calls `dlx_universe_search` recursively (backtracking), incrementing `k`.Here is the call graph for this

function:



Here is the caller graph for this function:



dlx_displayer_set()

```
dlx_solution_displayer dlx_displayer_set (
            Universe universe,
            dlx_solution_displayer displayer,
            void * data )
```

Setter of solution displayer.

Parameters

in	*universe*	Universe
in	*displayer*	Solution displayer to set.
in	*data*	Pointer to user defined and allocated data passed.

Returns

Solution displayer set by the previous call to dlx_displayer_set() (or `NULL` on first call).

The function pointed to by `displayer` passed as an argument, if set, is called by dlx_exact_cover_search() every time a solution is found.

dlx_universe_create() [1/3]

```
Universe dlx_universe_create (
            void  )
```

Initialises a new unbound universe.

Returns

universe

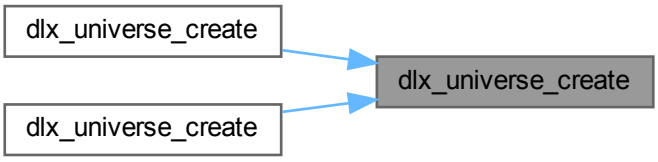
Note

The extension of the universe will match its subsets later added by dlx_subset_define(Universe universe, const char ∗subset_nar
or dlx_subset_define(Universe universe, const char ∗subset_name, unsigned long nb_elements, const char ∗some_elements[ ]

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

Here is the caller graph for this function:



dlx_universe_create() [2/3]

```
Universe dlx_universe_create (
            unsigned long nb_elements,
            const char * elements[] )
```

Initialises a new bound universe.

Parameters

in	*nb_elements*	Number of elements of the universe.
in	*elements*	Names of element of the universe.

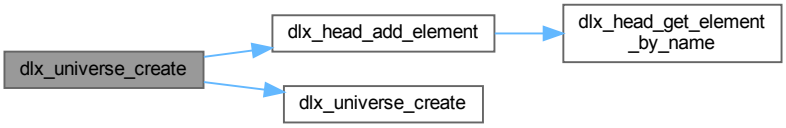
Returns

universe

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:



dlx_universe_create() [3/3]

```
Universe dlx_universe_create (
            const char * list_of_elements,
            const char * separators )
```

Initialises a new bound universe.

Parameters

in	*list_of_elements*	List of elements of the universe, separated by separators.
in	*separators*	List of accepted separators, terminated by \0. The separators argument specifies a set of bytes that delimit the tokens in the parsed string. The caller may specify different strings in separators in successive calls that parse the same string.

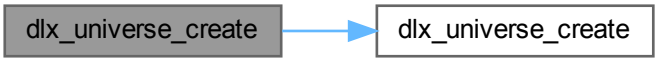
Returns

universe

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:



dlx_subset_define() [1/2]

```
int dlx_subset_define (
            Universe universe,
            const char * subset_name,
            unsigned long nb_elements,
            const char * some_elements[] )
```

Adds a subset to the universe.

Parameters

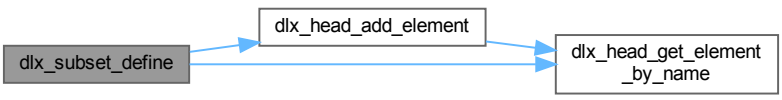
in	*universe*	Universe
in	*subset_name*	Name of the added subset
in	*nb_elements*	Number of elements of the universe contained in the subset.
in	*some_elements*	Names of elements of the universe contained in the subset.

Returns

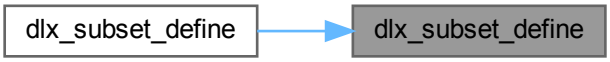
1 if added successfully, 0 otherwise.

The element of the universe and subsets containing this element of the universe are doubly linked as circular lists (`elementInPreviousSubsetContainingThisElementOfUnivers` and `elementIn↩NextSubsetContainingThisElementOfUnivers`).

The elements of a subset are doubly linked as circular lists (`previousElement` and `nextElement`).Here is the call graph for this function:



Here is the caller graph for this function:



dlx_subset_define() [2/2]

```
int dlx_subset_define (
            Universe universe,
            const char * subset_name,
            const char * list_of_some_elements,
            const char * separators )
```

Adds a subset to the universe.

Parameters

in	*universe*	Universe
in	*subset_name*	Name of the added subset
in	*list_of_some_elements*	List of elements of the universe contained in the subset, separated by separators.
in	*separators*	List of accepted separators, terminated by \0. The separators argument specifies a set of bytes that delimit the tokens in the parsed string. The caller may specify different strings in separators in successive calls that parse the same string.

Returns

1 if added sucessfully, 0 otherwise

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:

dlx_subset_require_in_solution()

```
int dlx_subset_require_in_solution (
            Universe universe,
            const char * subset_name )
```

Requires that a subset be included in any solution.

Parameters

in	*universe*	Universe
in	*subset_name*	Name of the required subset

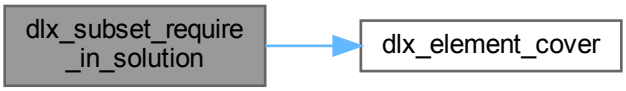
Returns

1 if successful, 0 otherwise.

Postcondition

User can therefore optionally add constraints to the searched solutions.

Here is the call graph for this function:



dlx_exact_cover_search()

```
unsigned long dlx_exact_cover_search (
            Universe universe,
            int one_only )
```

Searches for all exact cover solutions.

Parameters

in	*universe*	Universe
in	*one_only*	If set, searches for the first solution only.

Returns

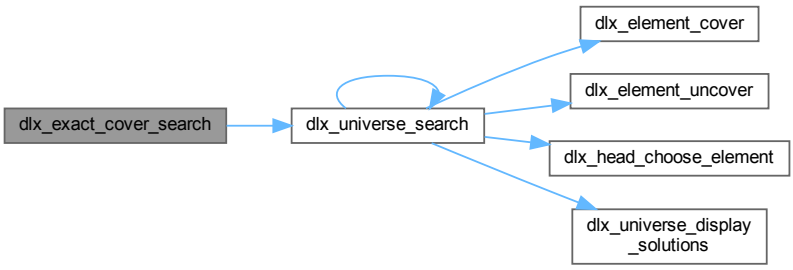
Number of solutions found.

Every time (or only the first time if `one_only` is set) a solution is found, the function `displayer` declared by a previous call to dlx_displayer_set(dlx_solution_displayer displayer) is called with three arguments:

- the identifier of the universe `universe`,

- the number of subsets included in the solution,
- the list of the names of the subsets included in the solution.

If dlx_displayer_set() was not called or was called with an argument equal to 0, solutions are displayed on standard terminal output. Here is the call graph for this function:



dlx_universe_destroy()

```
void dlx_universe_destroy (
            Universe universe )
```

Releases data used by the universe.

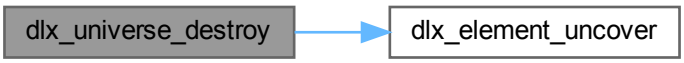
Parameters

in	*universe*	Universe

Precondition

Use dlx_universe_create(const char ∗elements, const char ∗separators) or dlx_universe_create(unsigned long nb_elements, co first.

Here is the call graph for this function:



5.1.4 Variable Documentation

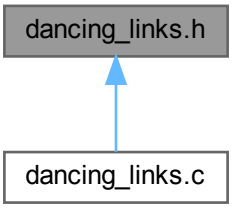
dlx_trace

```
int dlx_trace = 0
```

Flag to trace execution on standard error terminal (for debugging purpose only). Trace if set, keep quiet otherwise (default).

5.2 dancing_links.h File Reference

This graph shows which files directly or indirectly include this file:



Typedefs

- typedef struct universe ∗ **Universe**

  Objet Universe.
- typedef void(∗ dlx_solution_displayer) (Universe universe, unsigned long length, const char ∗const ∗solution, void ∗data)

Functions

- Universe dlx_universe_create (void) __attribute__((overloadable))
- Universe dlx_universe_create (const char ∗list_of_elements, const char ∗separators) __attribute__↩ ((overloadable))
- Universe dlx_universe_create (unsigned long nb_elements, const char ∗elements[ ]) __attribute__↩ ((overloadable))
- int dlx_subset_define (Universe universe, const char ∗subset_name, const char ∗list_of_some_elements, const char ∗separators) __attribute__((overloadable))
- int dlx_subset_define (Universe universe, const char ∗subset_name, unsigned long nb_elements, const char ∗some_elements[ ]) __attribute__((overloadable))
- int dlx_subset_require_in_solution (Universe universe, const char ∗subset_name)
- dlx_solution_displayer dlx_displayer_set (Universe universe, dlx_solution_displayer displayer, void ∗data)
- unsigned long dlx_exact_cover_search (Universe universe, int one_only)
- void dlx_universe_destroy (Universe universe)

Variables

- int dlx_trace

5.2.1 Detailed Description

Exact cover search interface. The aim is to find a list of subsets of the universe which all together, cover the whole universe, each subset being disjoint from the others. It's an exact cover of the universe.

5.2.2 Typedef Documentation

dlx_solution_displayer

```
typedef void(* dlx_solution_displayer) (Universe universe, unsigned long length, const char
*const *solution, void *data)
```

Solution displayer signature

Parameters

in	*universe*	Universe
in	*length*	Number of subsets in the solution
in	*solution*	List of the `length` names of the subsets in the solution.
in	*data*	Pointer to user defined and allocated data passed to `dlx_displayer_set()`.

5.2.3 Function Documentation

dlx_universe_create() [1/3]

```
Universe dlx_universe_create (
            void  )
```

Initialises a new unbound universe.

Returns

universe

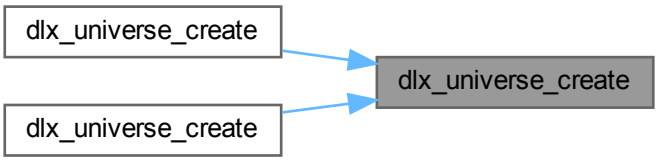
Note

The extension of the universe will match its subsets later added by dlx_subset_define(Universe universe, const char *subset_nar
or dlx_subset_define(Universe universe, const char *subset_name, unsigned long nb_elements, const char *some_elements[ ]

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

Here is the caller graph for this function:



dlx_universe_create() [2/3]

```
Universe dlx_universe_create (
            const char * list_of_elements,
            const char * separators )
```

Initialises a new bound universe.

Parameters

in	*list_of_elements*	List of elements of the universe, separated by separators.
in	*separators*	List of accepted separators, terminated by \0. The separators argument specifies a set of bytes that delimit the tokens in the parsed string. The caller may specify different strings in separators in successive calls that parse the same string.

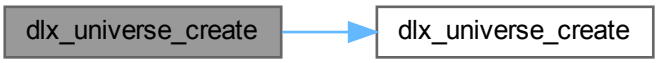
Returns

universe

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:



dlx_universe_create() [3/3]

```
Universe dlx_universe_create (
            unsigned long nb_elements,
            const char * elements[] )
```

Initialises a new bound universe.

Parameters

in	*nb_elements*	Number of elements of the universe.
in	*elements*	Names of element of the universe.

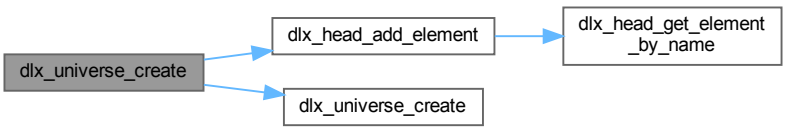
Returns

universe

Postcondition

User must call dlx_universe_destroy(Universe universe) later.

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:



dlx_subset_define() [1/2]

```
int dlx_subset_define (
            Universe universe,
            const char * subset_name,
            const char * list_of_some_elements,
            const char * separators )
```

Adds a subset to the universe.

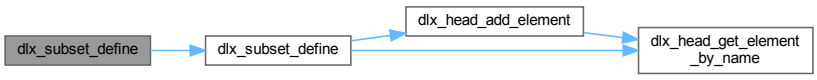
Parameters

in	*universe*	Universe
in	*subset_name*	Name of the added subset
in	*list_of_some_elements*	List of elements of the universe contained in the subset, separated by separators.
in	*separators*	List of accepted separators, terminated by \0. The separators argument specifies a set of bytes that delimit the tokens in the parsed string. The caller may specify different strings in separators in successive calls that parse the same string.

Returns

1 if added sucessfully, 0 otherwise

This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.Here is the call graph for this function:



dlx_subset_define() [2/2]

```
int dlx_subset_define (
            Universe universe,
            const char * subset_name,
            unsigned long nb_elements,
            const char * some_elements[] )
```

Adds a subset to the universe.

Parameters

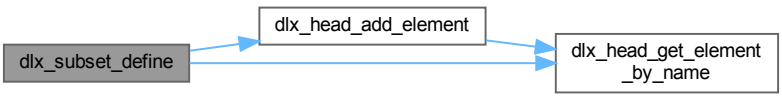
in	*universe*	Universe
in	*subset_name*	Name of the added subset
in	*nb_elements*	Number of elements of the universe contained in the subset.
in	*some_elements*	Names of elements of the universe contained in the subset.

Returns

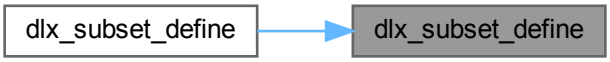
1 if added successfully, 0 otherwise.

The element of the universe and subsets containing this element of the universe are doubly linked as circular lists (`elementInPreviousSubsetContainingThisElementOfUnivers` and `elementIn↩NextSubsetContainingThisElementOfUnivers`).

The elements of a subset are doubly linked as circular lists (`previousElement` and `nextElement`).Here is the call graph for this function:



Here is the caller graph for this function:



dlx_subset_require_in_solution()

```
int dlx_subset_require_in_solution (
            Universe universe,
            const char * subset_name )
```

Requires that a subset be included in any solution.

Parameters

in	*universe*	Universe
in	*subset_name*	Name of the required subset

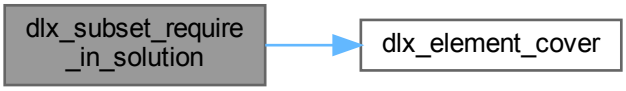
Returns

1 if successful, 0 otherwise.

Postcondition

User can therefore optionally add constraints to the searched solutions.

Here is the call graph for this function:



dlx_displayer_set()

```
dlx_solution_displayer dlx_displayer_set (
            Universe universe,
            dlx_solution_displayer displayer,
            void * data )
```

Setter of solution displayer.

Parameters

in	*universe*	Universe
in	*displayer*	Solution displayer to set.
in	*data*	Pointer to user defined and allocated data passed.

Returns

Solution displayer set by the previous call to dlx_displayer_set() (or `NULL` on first call).

The function pointed to by `displayer` passed as an argument, if set, is called by dlx_exact_cover_search() every time a solution is found.

dlx_exact_cover_search()

```
unsigned long dlx_exact_cover_search (
            Universe universe,
            int one_only )
```

Searches for all exact cover solutions.

Parameters

in	*universe*	Universe
in	*one_only*	If set, searches for the first solution only.

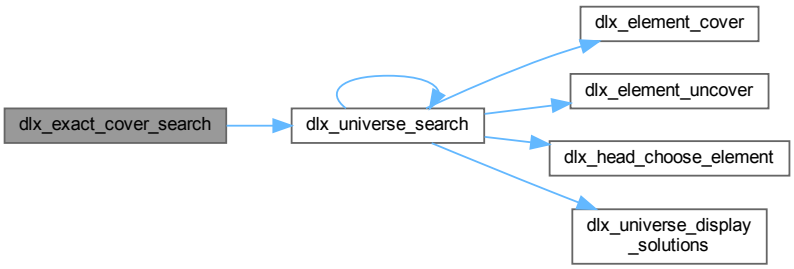
Returns

Number of solutions found.

Every time (or only the first time if `one_only` is set) a solution is found, the function `displayer` declared by a previous call to dlx_displayer_set(dlx_solution_displayer displayer) is called with three arguments:

- the identifier of the universe `universe`,
- the number of subsets included in the solution,
- the list of the names of the subsets included in the solution.

If dlx_displayer_set() was not called or was called with an argument equal to 0, solutions are displayed on standard terminal output. Here is the call graph for this function:



dlx_universe_destroy()

```
void dlx_universe_destroy (
            Universe universe )
```

Releases data used by the universe.

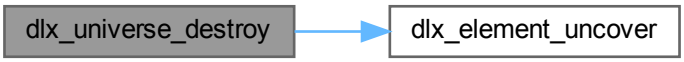
Parameters

in	*universe*	Universe

Precondition

Use dlx_universe_create(const char ∗elements, const char ∗separators) or dlx_universe_create(unsigned long nb_elements, co
first.

Here is the call graph for this function:



5.2.4 Variable Documentation

dlx_trace

```
int dlx_trace  [extern]
```

Flag to trace execution on standard error terminal (for debugging purpose only). Trace if set, keep quiet otherwise (default).

5.3 dancing_links.h

Go to the documentation of this file.

```
00001
00010 /*******
00011  * Copyright 2019 Laurent Farhi
00012  *
00013  *  This file is free software: you can redistribute it and/or modify
00014  *  it under the terms of the GNU Lesser General Public License as published by
00015  *  the Free Software Foundation, version 3.
00016  *
00017  *  This file is distributed in the hope that it will be useful,
00018  *  but WITHOUT ANY WARRANTY; without even the implied warranty of
00019  *  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
00020  *  GNU Lesser General Public License for more details.
00021  *
00022  *  You should have received a copy of the GNU Lesser General Public License
00023  *  along with this file.  If not, see <http://www.gnu.org/licenses/>.
00024  *****/
00025
```

```
#ifndef __DANCING_LINKS__
#define __DANCING_LINKS__
typedef struct universe *Universe;

Universe dlx_universe_create (void) __attribute__ ((overloadable));

Universe dlx_universe_create (const char *list_of_elements, const char *separators) __attribute__ ((overloadable));

Universe dlx_universe_create (unsigned long nb_elements, const char *elements[]) __attribute__ ((overloadable));

int dlx_subset_define (Universe universe, const char *subset_name, const char *list_of_some_elements, const char *separators)
    __attribute__ ((overloadable));

int dlx_subset_define (Universe universe, const char *subset_name, unsigned long nb_elements, const char *some_elements[])
    __attribute__ ((overloadable));

int dlx_subset_require_in_solution (Universe universe, const char *subset_name);

typedef void (*dlx_solution_displayer) (Universe universe, unsigned long length, const char *const *solution, void *data);

dlx_solution_displayer dlx_displayer_set (Universe universe, dlx_solution_displayer displayer, void *data);

unsigned long dlx_exact_cover_search (Universe universe, int one_only);

void dlx_universe_destroy (Universe universe);

extern int dlx_trace;

#endif
```

Index

dancing_links.c, 5
- DLX_DISPLAY_SOLUTION, 6
- dlx_displayer_set, 11
- dlx_element_cover, 8
- dlx_element_uncover, 9
- dlx_exact_cover_search, 16
- dlx_head_add_element, 7
- dlx_head_choose_element, 7
- dlx_head_get_element_by_name, 6
- dlx_subset_define, 14, 15
- dlx_subset_require_in_solution, 15
- dlx_trace, 17
- dlx_universe_create, 12, 13
- dlx_universe_destroy, 17
- dlx_universe_display_solutions, 9
- dlx_universe_search, 10

dancing_links.h, 18
- dlx_displayer_set, 23
- dlx_exact_cover_search, 24
- dlx_solution_displayer, 19
- dlx_subset_define, 21, 22
- dlx_subset_require_in_solution, 23
- dlx_trace, 25
- dlx_universe_create, 19, 20
- dlx_universe_destroy, 25

DLX_DISPLAY_SOLUTION
- dancing_links.c, 6

dlx_displayer_set
- dancing_links.c, 11
- dancing_links.h, 23

dlx_element_cover
- dancing_links.c, 8

dlx_element_uncover
- dancing_links.c, 9

dlx_exact_cover_search
- dancing_links.c, 16
- dancing_links.h, 24

dlx_head_add_element
- dancing_links.c, 7

dlx_head_choose_element
- dancing_links.c, 7

dlx_head_get_element_by_name
- dancing_links.c, 6

dlx_solution_displayer
- dancing_links.h, 19

dlx_subset_define
- dancing_links.c, 14, 15
- dancing_links.h, 21, 22

dlx_subset_require_in_solution
- dancing_links.c, 15
- dancing_links.h, 23

dlx_trace
- dancing_links.c, 17
- dancing_links.h, 25

dlx_universe_create
- dancing_links.c, 12, 13
- dancing_links.h, 19, 20

dlx_universe_destroy
- dancing_links.c, 17
- dancing_links.h, 25

dlx_universe_display_solutions
- dancing_links.c, 9

dlx_universe_search
- dancing_links.c, 10

element, 2

Exact cover search using dancing links Knuth's algorithm, 1

universe, 4